

РО 4.2. Разрабатывать серверные приложения и REST API с использованием Express.js

Интеграция REST API с клиентским приложением

Связываем frontend с Express API и выполняем CRUD из браузера

Результаты занятия

- ✓ Объяснять роль REST API между frontend и backend
- ✓ Отправлять HTTP-запросы из браузера с помощью fetch()
- ✓ Получать JSON и отображать данные в интерфейсе
- ✓ Отправлять POST-запросы из HTML-формы
- ✓ Выполнять PATCH и DELETE из клиентского приложения
- ✓ Проверять response.ok и обрабатывать ошибки API
- ✓ Организовывать клиентский код для работы с API

Учебные вопросы

- 1 Frontend и REST API** Как браузерное приложение взаимодействует с Express-сервером
- 2 Получение данных через GET** `fetch()`, `Response`, `JSON` и отображение товаров в `DOM`
- 3 Создание данных через POST** `HTML`-форма, `JSON.stringify()` и отправка `req.body`
- 4 Изменение и удаление данных** `PATCH` и `DELETE` из интерфейса пользователя
- 5 Ошибки и организация клиентского кода** `response.ok`, `try/catch`, `API`-функции и полный цикл frontend ↔ backend



Учебный вопрос 1. Frontend и REST API

Клиентское приложение становится потребителем нашего Product API

Что уже умеет backend

- Express принимает HTTP-запросы и выбирает нужный route.
- Controllers формируют HTTP-ответы.
- Services выполняют CRUD-операции с товарами.
- Validation middleware проверяет входные данные.
- Error handler формирует единообразные ответы об ошибках.
- Product API доступен через `/api/products` и `/api/products/:id`.

Что должен сделать frontend

- Отправить HTTP-запрос к нужному endpoint.
- Передать данные пользователя в формате, понятном API.
- Получить HTTP-ответ сервера.
- Преобразовать JSON-ответ в JavaScript-данные.
- Изменить DOM в соответствии с результатом.
- Показать пользователю успешный результат или ошибку.

Главная идея

Frontend не обращается к service или базе данных напрямую — он работает с backend через HTTP API.

Разделение frontend и backend

Frontend

- HTML, CSS и JavaScript
- Интерфейс пользователя
- DOM и события
- Формы и кнопки
- `fetch()` и обработка ответов

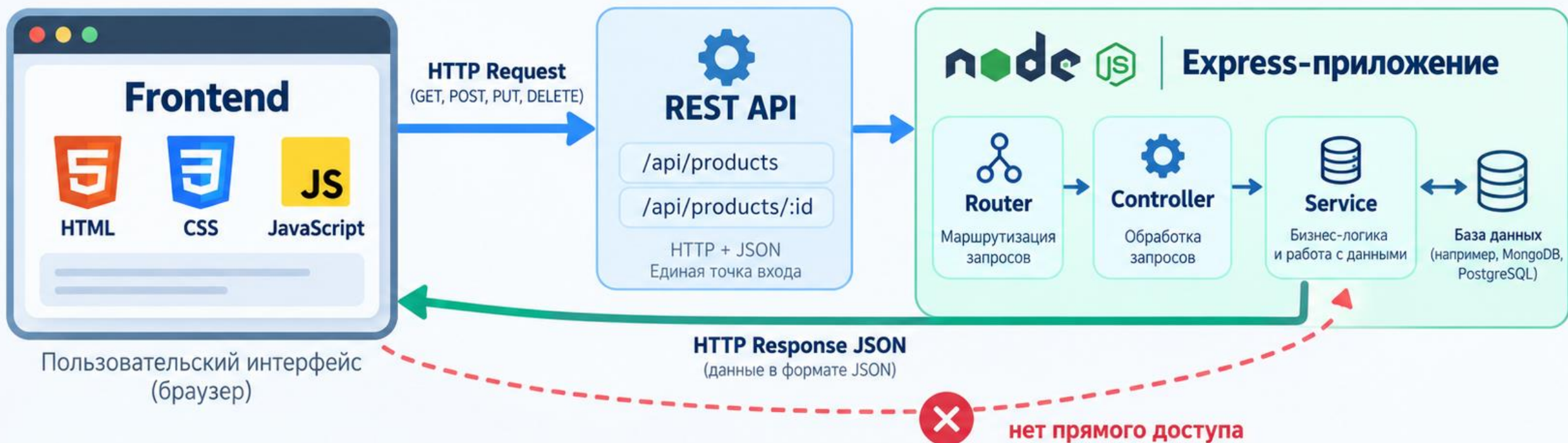
Backend

- Node.js и Express
- REST API
- Routes и Controllers
- Services
- Валидация и обработка ошибок

Frontend → REST API → Express-приложение



Разделение ответственности — чистая и понятная архитектура



Frontend может:

- Отправлять HTTP-запросы к REST API
- Получать данные в формате JSON
- Отображать данные пользователю



Frontend не может:

- Иметь прямой доступ к Service
- Запрашивать данные из базы напрямую
- Обходить маршруты и контроллеры



Результат:

- Четкое разделение слоев
- Безопасность и контролируемый доступ
- Легче поддерживать и развивать
- Соответствие принципам REST и MVC

Fetch API

- Fetch API — браузерный интерфейс для выполнения сетевых запросов из JavaScript и получения объекта Response.
- `fetch()` принимает URL запрашиваемого ресурса.
- По умолчанию выполняется GET-запрос.
- Для POST, PATCH и DELETE используются параметры второго аргумента.
- `fetch()` возвращает Promise.
- С `async/await` сетевой код читается как последовательность действий.

Первый запрос к Product API

```
const response = await fetch('/api/products');  
const products = await response.json();  
  
console.log(products);
```

Что важно

- Браузер отправляет GET /api/products.
- await ожидает получение объекта Response.
- response.json() асинхронно читает JSON-тело ответа.
- products становится обычным JavaScript-массивом.

Почему используем относительный URL

- Если frontend и API отдаются одним Express-приложением, у них один origin.
- Клиент может обращаться к API как `/api/products`.
- Не требуется жёстко записывать `localhost:3000` в каждом запросе.
- Код проще переносить между компьютерами и окружениями.
- В лабораторной frontend будет находиться в каталоге `public`.

Express отдаёт клиентские файлы

```
app.use(express.static('public'));  
app.use('/api/products', productRoutes);
```

Что важно

- `express.static('public')` публикует HTML, CSS и клиентский JavaScript.
- Браузер открывает страницу с того же сервера Express.
- Маршруты `/api/products` продолжают обслуживать REST API.
- Frontend и backend доступны через одно приложение.



Учебный вопрос 2. Получение данных через GET

Получаем массив товаров и превращаем JSON в элементы интерфейса

Путь GET-запроса из браузера

- 1 JavaScript вызывает fetch()** Клиент выполняет GET /api/products.
- 2 Express обрабатывает запрос** Product Router передаёт запрос controller и service.
- 3 API возвращает JSON** Сервер отвечает массивом объектов товаров.
- 4 Клиент читает response.json()** JSON преобразуется в JavaScript-массив products.
- 5 Frontend изменяет DOM** Для каждого товара создаётся элемент интерфейса.

Путь GET-запроса от fetch() до DOM

От запроса к данным и дальше — к интерфейсу пользователя



HTTP-запрос

Получаем данные с сервера



HTTP-ответ (JSON)

Данные приходят в виде JSON



JavaScript-данные

Преобразуем в массив объектов



DOM

Создаём элементы на странице

Один GET-запрос — и данные с сервера превращаются в интерактивный интерфейс!

Функция loadProducts

```
async function loadProducts() {  
  const response = await fetch('/api/products');  
  
  if (!response.ok) {  
    throw new Error('Failed to load products');  
  }  
  
  const products = await response.json();  
  renderProducts(products);  
}
```

Что важно

- fetch() выполняет запрос к API.
- response.ok проверяет успешный HTTP-статус.
- response.json() получает массив товаров.
- renderProducts() отвечает только за отображение.
- Сетевую работу и работу с DOM удобно разделять.

Объект Response

- `response.status` содержит числовой HTTP-код.
- `response.ok` равен `true` для успешных ответов 2xx.
- `response.json()` читает тело как JSON.
- Получение `Response` ещё не означает, что операция успешна.
- Например, ответ 404 также приходит как объект `Response`.

Главная идея

Для HTTP-ошибок
необходимо самостоятельно
проверить `response.ok` или
`response.status`.

Отображение товаров в DOM

```
function renderProducts(products) {  
  const list = document.querySelector('#products');  
  list.innerHTML = '';  
  
  for (const product of products) {  
    const item = document.createElement('li');  
    item.textContent = `${product.name} — ${product.price}`;  
    list.append(item);  
  }  
}
```

Что важно

- Функция получает обычный массив объектов.
- Список очищается перед повторным отображением.
- Для каждого товара создаётся элемент li.
- Данные API преобразуются в содержимое интерфейса.

Данные API становятся интерфейсом

Запрос: GET /api/products

```
[  
  { "id": 1, "name": "Keyboard",  
    "price": 15000 },  
  { "id": 2, "name": "Mouse",  
    "price": 8000 }  
]
```

Пояснение

- Frontend получает массив JSON и создаёт из него список или карточки товаров в DOM.

Загрузка товаров при открытии страницы

```
document.addEventListener('DOMContentLoaded', () => {  
  loadProducts();  
});
```

Что важно

- DOMContentLoaded выполняется после построения DOM.
- После загрузки страницы вызывается loadProducts().
- Frontend получает актуальные данные с сервера.
- Не нужно записывать товары непосредственно в HTML.



Учебный вопрос 3. Создание данных через POST

Данные HTML-формы превращаются в JSON-запрос к REST API

Что требуется для POST-запроса

- Получить значения из элементов формы.
- Сформировать JavaScript-объект товара.
- Вызвать `fetch()` с `method: 'POST'`.
- Передать заголовок `Content-Type: application/json`.
- Преобразовать объект в строку через `JSON.stringify()`.
- Обработать созданный объект или ошибку API.

Форма добавления товара

```
<form id="product-form">
  <input id="name" placeholder="Название" required>
  <input id="price" type="number" placeholder="Цена" required>
  <input id="category" placeholder="Категория" required>
  <label>
    <input id="inStock" type="checkbox">
    В наличии
  </label>
  <button type="submit">Добавить</button>
</form>
```

Путь данных формы при POST-запросе

Как данные из HTML-формы превращаются в новый ресурс через Express REST API

Клиент (браузер, фронтенд)

Заполняем форму и готовим данные

HTTP и JSON

Отправляем данные на сервер

Сервер (Express REST API)

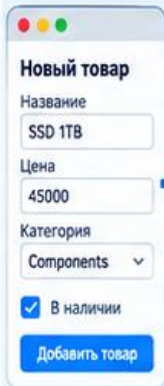
Обрабатываем запрос

Ответ и обновление UI

Получаем результат и обновляем интерфейс

1

HTML-форма



Пользователь заполняет форму в браузере.

2

Получение .value и .checked

```
const form = document.querySelector("#productForm");
const name = form.name.value;
const price = form.price.value;
const category = form.category.value;
const inStock = form.inStock.checked;
```

Считываем значения полей формы: .value для текстовых полей и .checked для чекбоксов.

3

Объект product

```
const product = {
  name: "SSD 1TB",
  price: 45000,
  category: "Components",
  inStock: true
};
```

Формируем JavaScript-объект с данными товара.

4

JSON.stringify (product)

```
const json = JSON.stringify(product);
```

Преобразуем объект в JSON-текст для отправки на сервер.

5

POST /api/products

```
POST /api/products
Content-Type: application/json
{
  "name": "SSD 1TB",
  "price": 45000,
  "category": "Components",
  "inStock": true
}
```

Отправляем HTTP POST-запрос с JSON-данными на сервер.

6

express.json()



Express middleware парсит JSON из тела запроса.

7

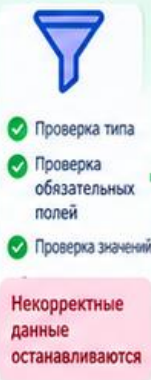
req.body

```
req.body = {
  name: "SSD 1TB",
  price: 45000,
  category: "Components",
  inStock: true
}
```

Распарсенные данные доступны в req.body.

8

Validation



Проверяем и фильтруем входящие данные.

9

Controller



Контроллер принимает данные, вызывает сервис и формирует ответ.

10

Service



Сервис сохраняет товар в базу данных.

11

201 Created

```
HTTP/1.1 201 Created
{
  "id": 16,
  "name": "SSD 1TB",
  "price": 45000,
  "category": "Components",
  "inStock": true
}
```

Сервер возвращает статус 201 Created с созданным ресурсом.

12

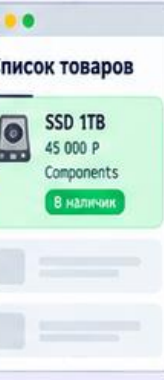
Созданный объект

```
{
  "id": 16,
  "name": "SSD 1TB",
  "price": 45000,
  "category": "Components",
  "inStock": true
}
```

Новый товар с присвоенным id создан в системе.

13

Обновление интерфейса



Интерфейс обновляется — новый товар появляется в списке без перезагрузки страницы.

Итоговая схема:



Форма



JavaScript-объект



JSON



REST API



Express



Новый товар



Обновлённый интерфейс



Результат:

данные из формы превратились в новый ресурс в базе данных, а интерфейс обновился!

Получаем данные формы

```
const product = {  
  name: document.querySelector('#name').value.trim(),  
  price: Number(document.querySelector('#price').value),  
  category: document.querySelector('#category').value.trim(),  
  inStock: document.querySelector('#inStock').checked  
};
```

Что важно

- value возвращает строковое значение поля.
- Для price выполняется явное преобразование в Number.
- checked возвращает boolean.
- Полученный объект соответствует структуре Product API.

Отправляем POST через fetch()

```
const response = await fetch('/api/products', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify(product)
});

const result = await response.json();
```

Что важно

- method определяет HTTP-метод POST.
- Content-Type сообщает серверу формат тела запроса.
- JSON.stringify() превращает объект в JSON-строку.
- express.json() на сервере снова превращает JSON в req.body.
- result содержит созданный товар или сообщение об ошибке.

Что происходит после submit

- 1 Пользователь отправляет форму** Событие submit перехватывается клиентским JavaScript.
- 2 Frontend формирует объект** Значения полей преобразуются в объект product.
- 3 fetch() отправляет POST** JSON передаётся в POST /api/products.
- 4 API валидирует и создаёт товар** Validation middleware проверяет данные, service выполняет Create.
- 5 Frontend получает результат** При успехе API возвращает 201 Created и созданный объект.
- 6 Интерфейс обновляется** Frontend повторно загружает список товаров или добавляет созданный объект в DOM.

Обработчик submit

```
form.addEventListener('submit', async (event) => {  
  event.preventDefault();  
  
  const product = getProductFromForm();  
  await createProduct(product);  
  form.reset();  
  await loadProducts();  
});
```

Что важно

- `preventDefault()` отменяет стандартную отправку HTML-формы.
- `getProductFromForm()` получает данные пользователя.
- `createProduct()` отправляет запрос к API.
- `reset()` очищает форму.
- `loadProducts()` обновляет интерфейс после изменения данных.



Учебный вопрос 4. Изменение и удаление данных

PATCH и DELETE вызываются непосредственно из действий пользователя

Как связать кнопку с конкретным товаром

- Каждая карточка товара содержит его id.
- Кнопке можно добавить data-id.
- Обработчик события получает id выбранного товара.
- По этому id формируется URL `/api/products/:id`.
- Один и тот же код работает для любого элемента списка.

Кнопки товара с data-id

```
<button class="edit-btn" data-id="15">
  Изменить
</button>

<button class="delete-btn" data-id="15">
  Удалить
</button>
```

Что важно

- data-id хранит идентификатор ресурса.
- JavaScript может получить его через `element.dataset.id`.
- id связывает DOM-элемент с конкретным ресурсом REST API.

PATCH-запрос из frontend

```
async function updateProduct(id, changes) {  
  const response = await fetch(`/api/products/${id}`, {  
    method: 'PATCH',  
    headers: {  
      'Content-Type': 'application/json'  
    },  
    body: JSON.stringify(changes)  
  });  
  
  if (!response.ok) {  
    throw new Error('Failed to update product');  
  }  
  
  return response.json();  
}
```

Что важно

- id включается в URL конкретного ресурса.
- changes содержит только изменяемые поля.
- PATCH выполняет частичное обновление.
- Функция возвращает обновлённый объект.

Редактирование товара через PATCH

Как клиентское приложение частично изменяет данные товара через REST API



PATCH = частичное обновление ресурса

1 Пользователь выбирает товар

В браузере пользователь находит товар и выбирает карточку с **id=15**.



SSD 1TB
ID: 15
45 000 ₽
Components

● В наличии

Пользователь просматривает список товаров и выбирает товар для редактирования.

2 Кнопка с data-id="15"

На карточке товара нажимает кнопку «Изменить».

Изменить

data-id="15"

Кнопка содержит атрибут **data-id="15"**, по которому фронтенд понимает, какой товар нужно изменить.

3 Frontend формирует объект изменений

Клиентское приложение создаёт объект только с изменяемыми полями (в нашем случае — цена).

JS JavaScript

```
const changes = {  
  "price": 42000  
};
```

Отправляем только те поля, которые нужно изменить. Остальные данные товара не передаются.

4 HTTP-запрос PATCH

Frontend отправляет **PATCH**-запрос на конкретный товар по его id.



HTTP-запрос

```
PATCH /api/products/15  
Content-Type: application/json  
{  
  "price": 42000  
}
```

Используется метод **PATCH** и передаются только изменяемые поля в формате JSON.

5 Product API (Express)

Сервер (Express API) получает запрос, проверяет данные, обновляет товар в базе данных.



Express API



✓ Проверяем id=15



✓ Валидируем данные



✓ Обновляем только указанные поля



✓ Сохраняем в БД

6 Сервер возвращает обновлённый объект

API возвращает полный объект товара с учётом изменений.



JSON-ответ

```
{  
  "id": 15,  
  "name": "SSD 1TB",  
  "price": 42000,  
  "category": "Components",  
  "inStock": true  
}
```

В ответе приходит обновлённый товар. Изменилось только поле **price**, остальные поля остались прежними.

7 Frontend обновляет интерфейс

Клиентское приложение обновляет карточку товара или повторно загружает список.



SSD 1TB
ID: 15
42 000 ₽
Components

● В наличии



Можно сразу обновить карточку на странице или выполнить повторную загрузку списка товаров.



Главная идея

PATCH = частичное обновление ресурса

Мы изменяем только одно поле (**price**), не отправляя весь объект товара.



Важно помнить

- ✓ Используем **PATCH** для частичных изменений
- ✓ Передаём только нужные поля
- ✓ Остальные поля товара не изменяются
- ✓ Сервер возвращает обновлённый объект
- ✓ Фронтенд обновляет интерфейс или перезагружает список



Итоговая схема:



Пользователь (браузер)



Frontend (JavaScript)



PATCH /api/products/15



Product API (Express)



JSON ответ



Обновлённый интерфейс

Изменение только цены

Запрос: PATCH /api/products/15

```
{  
  "price": 42000  
}
```

```
{  
  "id": 15,  
  "name": "SSD 1TB",  
  "price": 42000,  
  "category": "Components",  
  "inStock": true  
}
```

Пояснение

- Frontend отправляет только изменённое поле, а API возвращает актуальное состояние ресурса.

DELETE-запрос из frontend

```
async function deleteProduct(id) {  
  const response = await fetch(`/api/products/${id}`, {  
    method: 'DELETE'  
  });  
  
  if (!response.ok) {  
    throw new Error('Failed to delete product');  
  }  
}
```

Что важно

- URL содержит id удаляемого товара.
- Тело запроса для этой операции не требуется.
- После успешного удаления интерфейс необходимо синхронизировать с сервером.
- Можно повторно вызвать loadProducts().

Удаление товара из интерфейса

- 1 Пользователь нажимает «Удалить»** Кнопка содержит data-id выбранного товара.
- 2 Frontend получает id** JavaScript читает идентификатор через dataset.id.
- 3 Отправляется DELETE** fetch() выполняет DELETE /api/products/:id.
- 4 Backend удаляет ресурс** Route → Controller → Service выполняют операцию Delete.
- 5 Frontend обновляет список** После успешного ответа loadProducts() получает актуальное состояние коллекции.

Путь DELETE-запроса от кнопки до обновления DOM

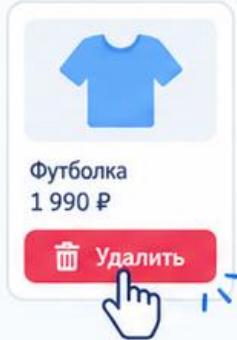


Frontend (браузер)

Пользователь нажимает кнопку, и в запросе передаётся id товара

1

Кнопка
«Удалить»



Пользователь нажимает кнопку «Удалить» на карточке товара.

2

data-id = 15

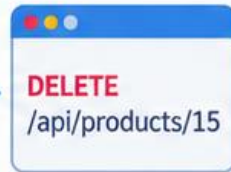
data-id="15"

Кнопка содержит атрибут **data-id** с идентификатором товара.

i Именно этот id передаётся дальше в запросе.

3

DELETE
/api/products/15



Frontend отправляет HTTP DELETE-запрос на конкретный ресурс с id 15.



Backend (Express API)

Запрос обрабатывается на сервере

4

Обработка
в Express



5

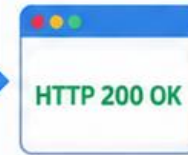
Удаление
из источника данных



Товар удаляется из базы данных (или другого источника данных).

6

Успешный
HTTP-ответ



Сервер возвращает успешный ответ о том, что товар удалён.



Frontend (браузер)

После успешного ответа обновляем список товаров

7

loadProducts()



Frontend повторно вызывает **loadProducts()**, чтобы загрузить актуальный список товаров.

8

Карточка
исчезает из DOM



Карточка товара с id 15 исчезает из DOM.
Список товаров синхронизируется с сервером.



После успешного DELETE frontend повторно загружает список товаров и DOM синхронизируется с сервером.
Это гарантирует, что пользователь видит актуальные данные.

Сервер — источник актуального состояния

- Frontend показывает данные, полученные от API.
- После Create, Update или Delete состояние на сервере изменяется.
- Интерфейс необходимо синхронизировать с новым состоянием.
- Самый простой учебный вариант — повторно вызвать GET /api/products.
- Позже можно оптимизировать интерфейс и изменять локальное состояние без полного повторного запроса.

Главная идея

Не следует считать DOM самостоятельным источником данных — сервер хранит состояние приложения.



Учебный вопрос 5. Ошибки и организация клиентского кода

Frontend должен обрабатывать не только успешные ответы

fetch() и HTTP-ошибки

- `fetch()` возвращает `Response` даже при HTTP-ответе 400 или 404.
- Поэтому одного `try/catch` недостаточно для проверки результата API.
- Нужно проверить `response.ok`.
- При ошибке можно прочитать JSON-сообщение сервера.
- После этого frontend показывает понятное сообщение пользователю.

Обработка ошибки API

```
async function request(url, options = {}) {
  const response = await fetch(url, options);
  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message || 'Request failed')
  }

  return data;
}
```

Что важно

- Response читается как JSON.
- `response.ok` определяет успешность HTTP-операции.
- Сообщение backend можно преобразовать в `Error`.
- Одна функция `request()` позволяет не повторять одинаковую проверку.

try/catch в интерфейсе

```
try {  
  await createProduct(product);  
  await loadProducts();  
  showMessage('Товар добавлен');  
} catch (error) {  
  showError(error.message);  
}
```

Что важно

- Успешный путь обновляет данные и интерфейс.
- Ошибка перехватывается в одном месте.
- Пользователь получает понятное сообщение.
- Техническую ошибку можно дополнительно записать в `console.error()`.

Состояния клиентского интерфейса

Loading

Запрос выполняется — показать загрузку или временно отключить кнопку.

Success

Операция выполнена — обновить DOM и показать результат.

Empty

Сервер вернул пустую коллекцию — показать сообщение «Товаров нет».

Error

Запрос завершился ошибкой — показать понятное сообщение пользователю.

Минимальная структура frontend

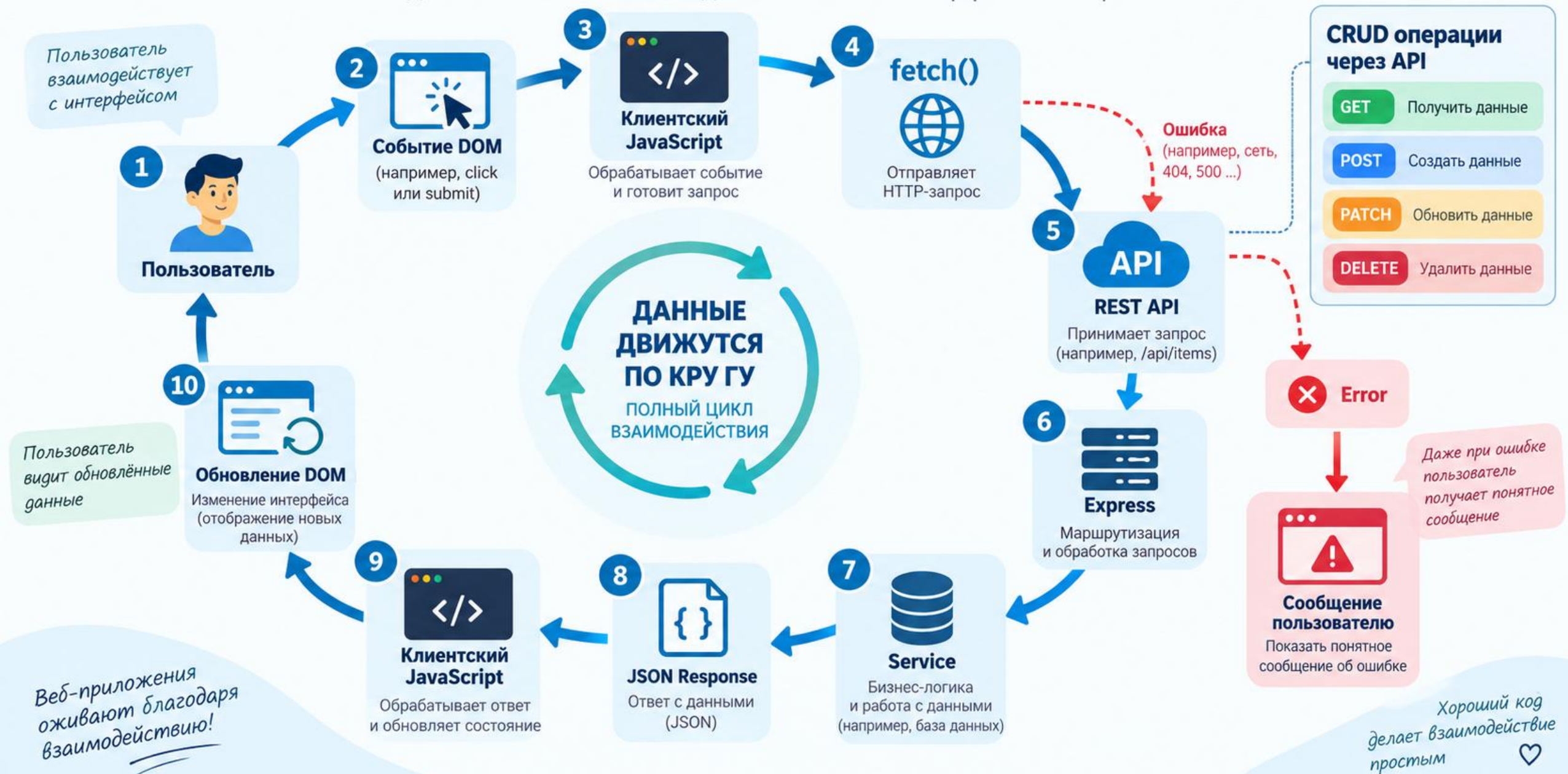
```
public/  
├── index.html  
├── css/  
│   └── style.css  
└── js/  
    ├── api.js  
    ├── ui.js  
    └── app.js
```

Разделение клиентского JavaScript

Файл	Ответственность
api.js	fetch(), GET, POST, PATCH, DELETE и работа с Response
ui.js	Создание DOM-элементов и сообщения пользователю
app.js	События страницы и координация API с интерфейсом

Полный цикл Frontend ↔ REST API ↔ DOM

От действия пользователя до обновления интерфейса и обратно



Если frontend и API работают на разных origin

- Origin определяется сочетанием протокола, хоста и порта.
- Например, localhost:5500 и localhost:3000 имеют разные origin из-за разных портов.
- Для cross-origin запросов браузер применяет правила CORS.
- Настройку CORS и безопасное взаимодействие frontend/backend подробно рассмотрим в РО 4.4.
- В текущей лабораторной frontend и API работают с одного Express-сервера.

Мини-практика: подключите страницу к Product API

Создайте страницу, которая загружает товары из REST API и позволяет добавить и удалить товар.

Задание

- Подключить каталог public через `express.static()`
- Создать HTML-список товаров
- Реализовать `loadProducts()` через GET `/api/products`
- Отобразить товары в DOM
- Создать форму добавления товара
- Отправить POST `/api/products`
- Добавить кнопки удаления с `data-id`
- Выполнить DELETE `/api/products/:id`
- Обработать `response.ok` и ошибки

Критерии проверки

- Товары загружаются с backend, а не записаны в HTML
- После POST новый товар появляется в интерфейсе
- DELETE удаляет выбранный товар
- После операций DOM соответствует данным API
- Ошибки API показываются пользователю

Контрольные вопросы

1. Какую роль REST API выполняет между frontend и backend?
2. Что возвращает функция `fetch()`?
3. Зачем вызывается `response.json()`?
4. Почему необходимо проверять `response.ok`?
5. Что делает `JSON.stringify()` при POST-запросе?

6. Зачем указывать Content-Type: `application/json`?
7. Как связать кнопку редактирования или удаления с `id` товара?
8. Почему после CRUD-операции интерфейс необходимо синхронизировать с сервером?
9. Чем отличаются обязанности `api.js` и `ui.js`?
10. Почему frontend на другом порту может потребовать настройки CORS?

Итоги занятия

1

Frontend взаимодействует с backend через HTTP-запросы к REST API.

2

`fetch()` позволяет выполнять GET, POST, PATCH и DELETE из браузерного JavaScript.

3

`response.json()` преобразует JSON-ответ сервера в JavaScript-данные.

4

HTML-форма может быть преобразована в объект и отправлена как JSON через POST.

5

`id` ресурса связывает элементы интерфейса с endpoints `/api/products/:id`.

6

`response.ok` необходимо проверять отдельно, потому что HTTP-ошибка не равна сетевой ошибке `fetch()`.

7

После CRUD-операций frontend должен синхронизировать DOM с актуальными данными сервера.

8

Разделение `api.js`, `ui.js` и `app.js` делает клиентский код понятнее и удобнее для развития.